

Executive Summary

Despite the rapid growth of sim-racing esports, there remains a shortage of rigorous, data-driven analytics conducted among professional and amateur esports teams. This study demonstrates how applying data science methods can reveal key driver behaviours that influence lap time through one of Formula One's most technically demanding segments.

Using extensive telemetry data from the Melbourne Albert Park Circuit's Turns 1 and 2 chicane, detailed data cleaning, preparation and exploratory data analysis was conducted. Several models were trained and evaluated, including XGBoost, LightGBM and Support Vector Regression (SVR), and performance success metrics obtained from these models were used to assess accuracy and generalisation. SVR emerged as the strongest model, achieving the lowest prediction error and explaining over 90 percent of the variance in lap time. Using this final model, differential evolution was applied to identify the optimal combination of braking, steering, and throttle inputs that minimised chicane time while maximising exit speed. A plausibility check using KNN confirmed that the generated optimal lap held close resemblance to real laps in the dataset. Hence, the results highlight that trigger points such as timing of throttle lift, steering initiation into Turn 1, and throttle re-application exiting Turn 2 most strongly affected performance, with a realistic target lap time of approximately 11.27 seconds throughout the sequence. While there were some notable constraints, overall our study demonstrates how data-driven analysis can guide performance optimization. These findings offer key metrics for refining training tools and techniques in sim racing performance improvement.

Introduction

Background

In elite motorsports, marginal differences in driver input, particularly during corner entry and exit, can have substantial effects on overall lap performance. Looking at recent analyses from the 2025 Australian Grand Prix, the difference of just a few tenths of a seconds through these opening corners were enough to shape the race outcome between teams like Ferrari and McLaren (Carter, 2025). This level of sensitivity underscores the importance of precision in driver mechanics, and the ability to optimise the sequence of control actions through complex turning zones.

The Albert Park Circuit in Melbourne, one of Formula One's most distinctive racing circuits, is particularly known for its combination of high-speed straights and tight technical sections, challenging both driver precision and vehicle setup. One of the circuit's most defining features is the chicane at Turns 1 and 2. Success through this section sets up a driver's acceleration into Turn 3, which often becomes a decisive overtaking zone during the Australian Grand Prix.

Related Works

Recent research has increasingly applied data-driven and machine learning methods to motorsport telemetry. Studies have explored diverse topics ranging from human-autonomous driving interactions (Remonda et al., 2021) and performance clustering in sim racing environments (Adam et al., 2024), to dynamic modelling of driver-vehicle interactions for strategy optimisation (Tonoli, 2024). Such works collectively highlight the analytical potential of telemetry data. Yet, they also highlight a persistent gap in the lack of granular, segment-level analysis focused on complex track features such as chicanes, where time-critical driver decisions are most influential. Hence, our project contributes to the current literature by building upon their existing methodologies. We also draw upon these papers to inform our analytical framework and to justify our choice of models.

Project Aim

In our project, we aim to determine the optimal combination of driver trigger points through Turns 1 and 2 of the Albert Park Circuit to maximise exit speed and minimise time to a fixed reference point. By modelling how variation in braking, turning and throttle application interact, we seek to identify data-driven strategies that could enhance overtaking opportunities and overall lap consistency in simulated F1 racing.

Methodology

Plan

To tackle this complex task, we divided it into smaller, manageable components and developed a plan based on established data science techniques. The planned workflow is outlined below.

1. Choose a Dataset
2. Background Research
3. Exploratory Data Analysis
4. Data Cleaning
5. Modelling, Feature Selection, and Hyperparameter Tuning
6. Results
7. Model Optimization

Key Assumptions

For this task, we decided to use the data product created by group 2 as it was well-cleaned, already aggregated into laps, and included feature engineered terms. But in order to complete the task with the dataset, we had to make various assumptions. This was to ensure that we produced a fair and behavioural focused model that reflected driver technique.

Independence of Laps

A key assumption we made is that each lap is an independent observation, and that drivers do not significantly vary their racing path across laps. This assumption is required as the dataset does not include information regarding tyre wear, fuel load, track evolution, or session context which would influence performance. As a result, the model explains behavioural differences in lap time rather than long-run session effects.

Data Accuracy

Given the time constraints and project workload, we assumed that any part of the data not cleaned by us (as outlined in the EDA and Data Cleaning section) was correct. We made this assumption to ensure that the project could be completed by the deadline.

Fastest Speed Exiting the Chicane

The objective of this task is to optimise performance through the chicane by minimising time and maximising exit speed. However, given the complexity of optimising various factors simultaneously, we assume the lap with the fastest time will also deliver the highest realistic exit speed. Consequently, we assume that the exit speed of the recommended lap represents the maximum achievable speed when exiting the chicane.

EDA and Data Cleaning

THE vs THS

As we were unfamiliar with the new dataset, we did some general exploratory data analysis and created some visualisations to develop an understanding of its structure and key features. Upon inspection, a significant error was discovered.

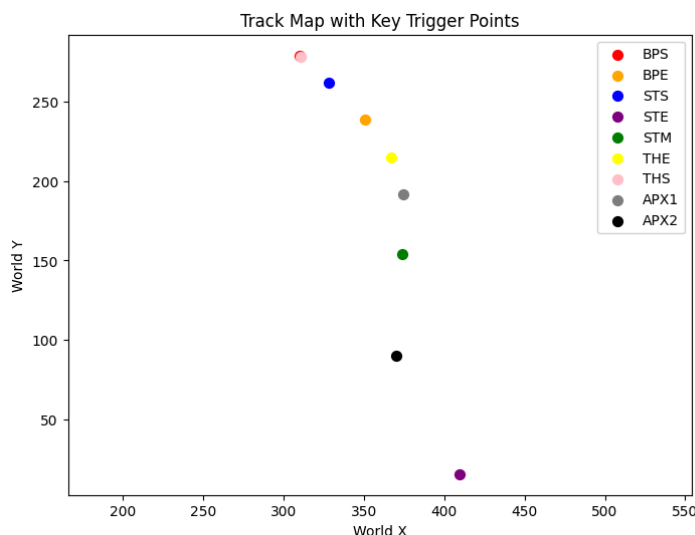


Figure 1. The track map and key trigger points for a specific lap (lap_id = '10021698832789871149_10')

Looking at Figure 1, which presents a track map outlining the key trigger points, the positions of the throttle end (THE - Yellow), and throttle start (THS - Pink) do not align with expectations. In the context of chicane navigation, the throttle would typically end upon entering the chicane, and then resume after the vehicle exits, which is the opposite of the illustration in Figure 1. This discrepancy suggests that the current labelling of THE and THS in the dataset has been reversed.

To verify this hypothesis, we compared the values of THE and THS using boxplots. As shown in Figure 2, the 'CURRENTLAPTIMEINMS' for the throttle start (THS) is smaller than that of the throttle end (THE). Since THE must occur before THS - the driver releasing the throttle to enter the chicane and pressing it again during exit - the pattern in Figure 2 suggests that the values in these columns may have been swapped. This mistake is also highlighted by Figure 3, where the 'LAPDISTANCE' variable, incorrectly suggests that the THS occurs before THE.

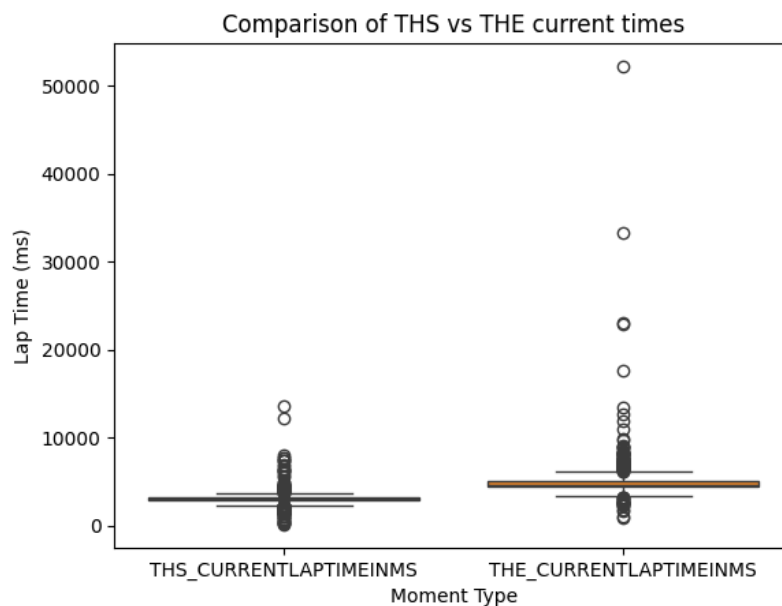


Figure 2. A comparative boxplot of 'current lap time in ms' values between throttle start (THS) and throttle end (THE).

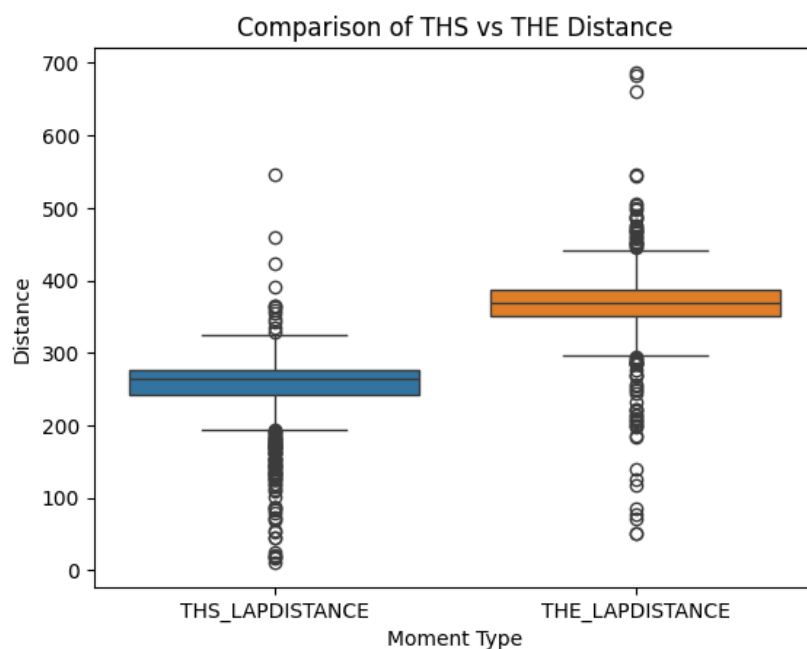


Figure 3. A comparative boxplot of 'lap distance' values between throttle start (THS) and throttle end (THE).

Steering whilst entering the chicane should be relatively straight, and greater steering is required when turning out, so, the THE angle should be smaller than the THS steering angle. However, the dataset shows otherwise (Appendix Figure 4), highlighting the need for the swap.

When examining the brake, throttle, and speed variables, it's important to consider that THE is the moment the throttle initially ends - this means that throttle should be relatively high, brake low, and speed relatively high. Conversely, for the THS - which marks the resumption of the throttle - brake should be high, throttle low and speed should be low. Upon examining their comparative box plots (Appendix Figure 1-3), it is evident that the expected pattern is not reflected by the dataset, suggesting an error. All variables that were explored indicate a need to swap the values of THE and THS, as can be seen in Table 1.

Variable	Expectation	Observation	Result
_currenttimeinms	THS > THE	THS < THE	Need to swap
_SPEED	THS < THE	THS > THE	Need to swap
_BRAKE	THS > THE	THS < THE	Need to swap
_THROTTLE	THS < THS	THS > THE	Need to swap
_lapdistance	THS > THE	THS < THE	Need to swap
_STEER	THS > THE	THS < THE	Need to swap

Table 1. Summary of all the evidence found from comparative boxplots that suggest issues in the dataset between throttle start (THS) and throttle end (THE).

Removal of Outlier Laps

Outliers in lap time can distort model training by exaggerating errors, causing the model to learn patterns that do not reflect realistic driving behaviour. Since our objective is to understand how driver controlled inputs affect lap time at the chicane, it was important to remove laps that were irregular. Lap times that were slower than the driver's typical pace could be due to a result of game errors (e.g. glitches) or cars going off track. To address this, we applied the Interquartile Range (IQR) method to the target variable to identify outlier laps. Any laps exceeding the upper bound were treated as outliers and removed.

Handling Null Values

The dataset contained rows with null values across key features. Initially, we considered imputing these values using the mean or median. However, this approach violates our assumption of lap independence and we risk assigning 'good' values to otherwise poor laps, potentially confusing the model. We also explored methods of imputing the values based on similar laps, but this proved to be too complex. Ultimately, we decided that removing the rows with null values would be the best option.

Evaluating Performance

We will use a combination of the following metrics to evaluate model performance. Since each measure captures a different aspect of model performance, optimising across all metrics will help us to determine the best model.

Mean Squared Error (MSE)

Mean Squared Error (MSE) is the average of the squared difference between the predicted and actual values (Frost 2025). It quantifies the overall error made by the model and is defined by the following formula:

$$MSE = \frac{\sum (y_i - \hat{y}_i)^2}{n}$$

Where:

- n - the number of observations
- y_i - the actual value
- $\hat{y}_i$ - the predicted value

A lower MSE indicates a more accurate model, while a higher MSE suggests that the model's predictions deviate significantly from the actual values. Since the errors are squared, large mistakes are heavily penalised by the model, and as such this metric is highly sensitive to outliers and extreme values.

Root Mean Squared Error (RMSE)

The Root Mean Squared Error (RMSE) is the square root of the average squared difference between the model's predicted and actual values (Frost 2025). It measures model performance by capturing the standard deviation of the residuals (error). The formula for this metric is given below.

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Where:

- n - the number of observations
- y_i - the actual value
- $\hat{y}_i$ - the predicted value

A lower RMSE value indicates a better model, as it shows that the predictions are closer to the actual data. Unlike MSE, RMSE retains the same units as the original target variable, making its results more interpretable. This metric responds to fluctuations in predictions, which can highlight when model performance changes. However, like MSE, the metric is sensitive to large errors and outliers.

Mean Absolute Error (MAE)

Mean Absolute Error (MAE) evaluates model performance by calculating the average of the absolute differences between the predicted values and actual values (Frost 2025). The formula for the MAE is given below.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where,

- n - the number of observations
- y_i - the actual value
- $\hat{y}_i$ - the predicted value

A low MAE indicates a more accurate model, as the predicted values are closer to the actual values. Like RMSE, MAE values are expressed in the same units as the target variable, making it more interpretable than other metrics. Unlike the aforementioned metrics, MAE treats all errors equally and as such is more robust to outliers.

Adjusted R-squared

The adjusted R^2 is a modified version of R^2 that accounts for the number of predictions in the model (Frost 2025). It penalises models that include too many irrelevant predictions, providing a more accurate measure of model performance. The metric is defined by the following formula:

$$Adjusted R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

Where,

- R^2 - is the sample R^2
- N - is the sample size
- p - is the number of independent variables

R^2 ranges from 0 to 1, with higher values indicating a better model. Additionally this metric can be used to compare the effectiveness of models with different numbers of predictors.

Models

Feature Selection

The original dataset contained approximately 255 features. To ensure the model's output aligns with our objective, we only retained driver controlled inputs (Figure 4). These are features that the driver can directly influence while on the track, rather than outcomes of other variables (e.g. yaw, pitch and roll). For different models, further feature selection may have been conducted as means to improve the model's prediction.

```
driver_input = ['BPS_SPEED', 'BPS_THROTTLE', 'BPS_STEER', 'BPS_BRAKE', 'BPS_CURRENTLAPTIMEINMS',
               'BPS_LAPDISTANCE', 'BPS_WORLDPOSITIONX', 'BPS_WORLDPOSITIONY', 'BPE_SPEED', 'BPE_THROTTLE',
               'BPE_STEER', 'BPE_BRAKE', 'BPE_CURRENTLAPTIMEINMS', 'BPE_LAPDISTANCE', 'BPE_WORLDPOSITIONX', 'BPE_WORLDPOSITIONY',
               'THS_SPEED', 'THS_THROTTLE', 'THS_STEER', 'THS_BRAKE', 'THS_CURRENTLAPTIMEINMS', 'THS_LAPDISTANCE', 'THS_WORLDPOSITIONX',
               'THS_WORLDPOSITIONY', 'THE_SPEED', 'THE_THROTTLE', 'THE_STEER', 'THE_BRAKE', 'THE_CURRENTLAPTIMEINMS',
               'THE_LAPDISTANCE', 'THE_WORLDPOSITIONX', 'THE_WORLDPOSITIONY', 'STS_SPEED', 'STS_THROTTLE', 'STS_STEER', 'STS_BRAKE',
               'STS_CURRENTLAPTIMEINMS', 'STS_LAPDISTANCE', 'STS_WORLDPOSITIONX', 'STS_WORLDPOSITIONY', 'STM_SPEED', 'STM_THROTTLE',
               'STM_STEER', 'STM_BRAKE', 'STM_CURRENTLAPTIMEINMS', 'STM_LAPDISTANCE', 'STM_WORLDPOSITIONX', 'STM_WORLDPOSITIONY',
               'STE_SPEED', 'STE_THROTTLE', 'STE_STEER', 'STE_BRAKE', 'STE_CURRENTLAPTIMEINMS', 'STE_LAPDISTANCE', 'STE_WORLDPOSITIONX',
               'STE_WORLDPOSITIONY', 'Target_CURRENTLAPTIMEINMS']
```

Figure 4: The Driver Controlled Inputs in the Dataset

We also found that cleaning different the 'CURRENTLAPTIMEINMS' attributes of different prefixes (BPS, STS, BPE, STM, STE, THE, THS), could either improve or deteriorate our model's predictive accuracy. By using a modified best-subset style algorithm, the upper IQR removal was applied to various combinations of 'CURRENTLAPTIMEINMS'. While combinations of features affected models differently, this cleaning process ultimately resulted in a net improvement across all evaluation metrics.

Linear Regression

Simple Linear Regression

Simple Linear Regression (SLR) is a fundamental parametric technique that models the relationship between a continuous dependent variable y and one or more independent variables $X_1, X_2, \dots, X_n$. The model is expressed as:

$$\hat{y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

where $\beta_0, \beta_1, \dots, \beta_n$ are the coefficients learned from the training data, and ϵ represents the error term quantifying the numerical margin of error. Although a simple model, SLR is highly interpretable and

computationally efficient, making it a useful baseline in our analysis. However it is important to note that SLR is easily susceptible to bias and overfitting and is limited in its application due to its various assumptions.

After the SLR model was fitted to the training data, model parameters were estimated and predictions generated on the test set, producing the key performance metrics below (Figure 5).

```
MSE: 2915701.85
RMSE: 1707.54
MAE: 982.52
R²: 0.5083
Adjusted R²: 0.3247
```

Figure 5: Performance Metrics of the final SLR model

Overall, performance metrics for SLR were the largest (and poorest) across all our models. The model achieved a moderate adjusted R^2 score (0.32), indicating that it explained approximately 32% of the variance in lap time. However, both MSE and RMSE values are relatively high, suggesting that there was substantial prediction error. These results confirm the model's limited capability to capture the underlying non-linear relationships, and reinforce that the assumption linearity does not hold for this dataset. Realistically, lap times are influenced by non-linear dependencies and interactions between features such as throttle position, steering angle, and vehicle speed, which SLR cannot capture. Nevertheless, it serves as an important baseline model to benchmark the performance of our complex models.

Elastic Net Regression

Elastic Net Regression is a hybrid regularisation technique that combines the power of both Lasso and Ridge by introducing a weighted penalty that balances L1 and L2 regularisation. It addresses limitations presented by each individual method (Lasso's tendency to exclude correlated variables entirely; Ridge's inability to perform feature selection), by jointly penalising the model coefficients. To leverage these advantages and obtain a more stable and interpretable model, we decided to include Elastic Net Regression in our analysis.

The Elastic Net objective function is defined as:

$$\frac{1}{2n_{\text{samples}}} \|y - Xw\|_2^2 + \alpha l_{1_ratio} \|w\|_1 + \frac{1}{2} \alpha (1 - l_{1_ratio}) \|w\|_2^2$$

where α controls the overall strength of regularisation, and $\lambda = l_{1_ratio}$ determines the relative contribution of Lasso (L1) and Ridge (L2) penalties.

For our dataset, an initial Elastic Net model was implemented, using parameters $\alpha = 0.5$ and $l_{1_ratio} = 0.7$. The model was trained on the scaled training dataset, and evaluated on the test set. This configuration achieved moderate performance improvements over the SLR baseline with an MSE of 1996991.3501 and adjusted R^2 of 0.5374, suggesting that the current parameters did not fully optimize the trade-off between bias and variance. To refine performance, hyperparameter tuning was conducted using ElasticNetCV, which performs K-fold cross validation ($k = 5$) to identify the optimal regularisation parameters. The search range for α was widened substantially, to $[-4, 3]$ across 200 log-spaced points, to ensure that the global minimum of the cross-validated error was captured. The optimal hyperparameters identified (in Figure 6) indicated that the model leaned heavily toward L1 regularisation, suggesting that sparse feature selection improved performance by reducing overfitting and removing redundant predictors.

```
Optimal alpha: 9.884959046625585
Optimal l1_ratio: 0.99
```

Figure 6: Optimal tuning hyperparameters for ElasticNetCV

Using the above hyperparameters, the final Elastic Net model was retrained on the full training dataset and evaluated the scaled test data, producing the following performance metrics:

```

Final Tuned Elastic Net Evaluation:
MSE: 1987021.4904
RMSE: 1409.6175
MAE: 879.4120
R²: 0.6649
Adjusted R²: 0.5398

```

Figure 7: Performance Metrics of the final Elastic Net model

Compared to SLR, the tuned Elastic Net model delivered substantial improvement across all evaluation metrics. This model achieved a 30.8% improvement in explanatory power (R^2) and a 66.3% improvement in adjusted R^2 , indicating that Elastic Net captured considerably more of the underlying variability in lap performance. Furthermore, RMSE decreased by 17.4%, MSE by 31.9%, and MAE by 10.5%, reflecting more accurate and stable predictions. This comparison demonstrates that regularisation techniques enhance model generalisation by penalising overly complex coefficient patterns, thereby better capturing the underlying structure of the data.

K-Nearest Neighbour Regressor

K-Nearest Neighbour Regressor (KNNR) is a non-parametric model that makes predictions based on the closest 'k' data points. It relies on the assumption that data points with similar features will have similar target values. However, as the dataset grows, K-NN can become inefficient, potentially compromising model performance. Therefore, selecting appropriate hyperparameters, such as the distance metric and the optimal value of 'k' is crucial.

Telemetry data is highly non-linear, with small changes in key inputs (e.g. speed) resulting in significant effects on outputs (final lap time). K-NNR is well suited for this task due to its non-parametric nature, which allows it to capture complex, non-linear relationships. Moreover, K-NNR's core assumptions - similar inputs will yield similar outputs - align with the idea that two laps where drivers exhibit similar behaviours are likely to have similar final times. As such, we decided to test this model.

Prior to modelling, the data underwent cleaning as outlined in the 'Feature Selection'. Removing outliers in the 'CURRENTLAPTIMEINMS' attribute for 'STM', 'STE' and 'THS' improved the MSE and, consequently, overall model performance (Figure 8). After training the initial K-NNR model, hyperparameter tuning was performed using 'GridSearchCV', which identified 6 neighbors and a distance based metric as the optimal parameters.

```

=== Best Result for KNNR ===
Best prefixes to clean: ('STM', 'STE', 'THS')
Lowest MSE: 204541.83
RMSE: 452.26
MAE: 297.88
R²: 0.5695
Adjusted R²: 0.1060

```

Figure 8: Subset selection results for KNN showing the moments that require cleaning

The model was then retrained using the optimal parameters, producing the performance metrics presented below. The KNNR model outperforms the simpler linear models, indicating that the dataset contains a substantial amount of non-linearities that the model was able to capture. Despite this, the model performs poorly, with a high RMSE and MSE, and an R^2 of 0.03, meaning the model only explains about 3% of the variance in the target variable.

Our team acknowledges KNNR's computational limitations, since its processing requirements increase significantly as the dataset grows. Given the high dimensionality of our dataset and the model's relatively poor performance, our group concluded that KNNR might not be appropriate for our project. As such, further feature selection and tuning was not conducted.


```

Lowest MSE: 219623.44
RMSE: 468.64
MAE: 312.08
R²: 0.5274
Adjusted R²: 0.0372

```

Figure 9: Performance Metrics of the final KNNR model

Random Forest

Random Forest is an ensemble learning method that combines the predictions of multiple decision trees to produce a stable and accurate model. Each tree learns different patterns in the telemetry by training on a random bootstrap sample, which reduces the influence of noisy or biased trees on the final prediction. Splits are selected to minimise prediction error, which allows the model to capture complex relationships in braking, throttle, steering, speed, and other driver-controlled behaviours.

Random forest has been used in similar studies to analyse complex datasets and emphasise the importance of rigorous model evaluation. In particular, findings from 'Formula 1 Race Winner Prediction Using Random Forest and SHAP Analysis' (2025) demonstrated the effectiveness of Random Forest in predicting Formula 1 race winners with high-dimensional, interrelated features. Our dataset also involved a large number of correlated behavioral variables collected across multiple track segments. This highlights the suitability of Random Forest for our project as it is robust to noise and multicollinearity.

Before modelling, only driver controlled features - brake, throttle, steering, speed, and positional descriptors - were retained, ensuring the model learns from inputs that are directly controlled by the driver. As all driver controlled features are required to fulfil the aim, no further feature selection was performed. Hyperparameters such as the number of trees were tuned using GridSearchCV to balance stability and computational cost.

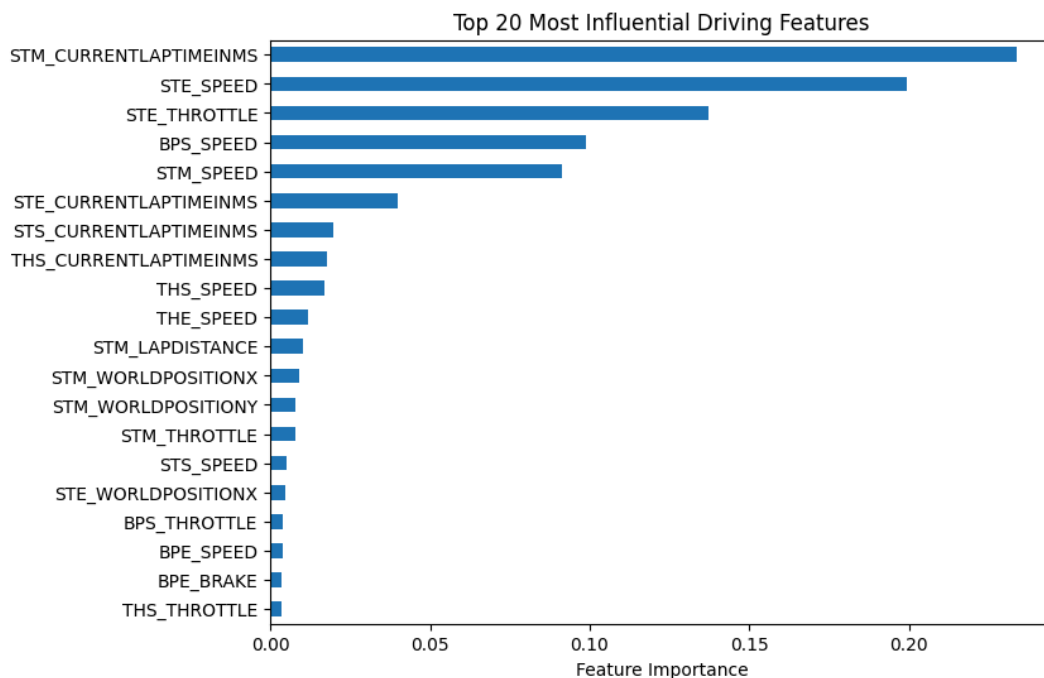


Figure 10: Top 20 Most Influential Driving features with Random Forest

Feature importance revealed that features relating to steering, braking, throttle and speed were the most influential in determining lap time differences. Specifically, STM_CURRENTLAPTIMEINMS, STE_SPEED, and STE_THROTTLE indicate that mid-lap pace and corner-exit behavior are the strongest determinants of lap time instead of isolated peak inputs. Initial braking speed and midpoint steering speed was also important

in determining lap time performance. Overall, the model highlights that faster laps arise from managing speed, throttle application, and vehicle stability through mid-corner and exit phases.

Support Vector Regression

Support Vector Regression (SVR) is a machine learning algorithm that finds a function to fit the data while keeping predictions within a specified margin of error. It can model both linear and non-linear relationships by using different kernel functions, allowing it to adapt to complex patterns in the dataset. Additionally the model incorporates regularization parameters that help to prevent overfitting thus improving model generalization.

SVR was seen in 'Data-Driven Vehicle Performance Optimization for Formula Student Racing' by Lal Akin (2024). In the paper, the team used SVR to analyse simulated racing data, with the aim of providing the student racing team at Politecnico di Torino with valuable racing insights, similar to our objective. Although SVR performed the poorest in their analysis, our team chose to test the model in our project due to its ability to capture both linear and non-linear relationships and its efficient handling of high-dimensional data.

Initial feature screening showed that removing outliers in the 'CURRENTLAPTIMEINMS' attribute for 'BPS', 'STM', 'STE' and 'THS' improved the MSE and overall model performance (Figure 11).

```
=== Best Result for SVR ===
Best prefixes to clean: ('BPS', 'STM', 'STE', 'THS')
Lowest MSE: 433294.03
RMSE: 658.25
MAE: 484.55
R2: 0.0074
Adjusted R2: -1.2852
```

Figure 11: Subset selection results for SVR showing the moments that require cleaning

After the initial training, hyperparameter tuning was performed using 'GridSearchCV', which identified the optimal parameters as outlined below (Figure 12). A linear kernel was identified as optimal, indicating that the relationship between feature engineered driver inputs and lap time is approximately linear. The error tolerance (ϵ) has a value of 100 means that the model will not be penalised if its predicted lap time is within 100ms of the actual time. The C value controls the penalty applied to wrong predictions; with a large value of C, errors are penalised heavily, pushing the model to fit more closely to the actual lap times.

```
Best parameters: {'C': 1000, 'epsilon': 100, 'kernel': 'linear'}
Best score: 34936.205396327525
```

Figure 12: The optimal hyperparameters of SVR as identified by GridSearch

The model was then retrained using the tuned hyperparameters, outperforming the other models. SVR achieved the best performance across all metrics, with an RMSE of 98.58ms and an adjusted R² score of 92.39%. Figure 13 shows the key features SVR used to predict the lap time. STE, which represents the steering angle returning to neutral after a turn, was the most important action, with metrics associated with it occupying the top three positions. This highlights the significance of steering in determining time. Overall, attributes related to steering (STM & STS), throttle (THS & THE) and braking (BPS & BPE), were the most important moments. This suggests that the timing, location, and intensity of steering, throttle and braking throughout different sections of the turn are the most critical factors in determining a driver's final lap time. Since information about all features is essential for generating a recommended lap, no additional feature selection was performed.

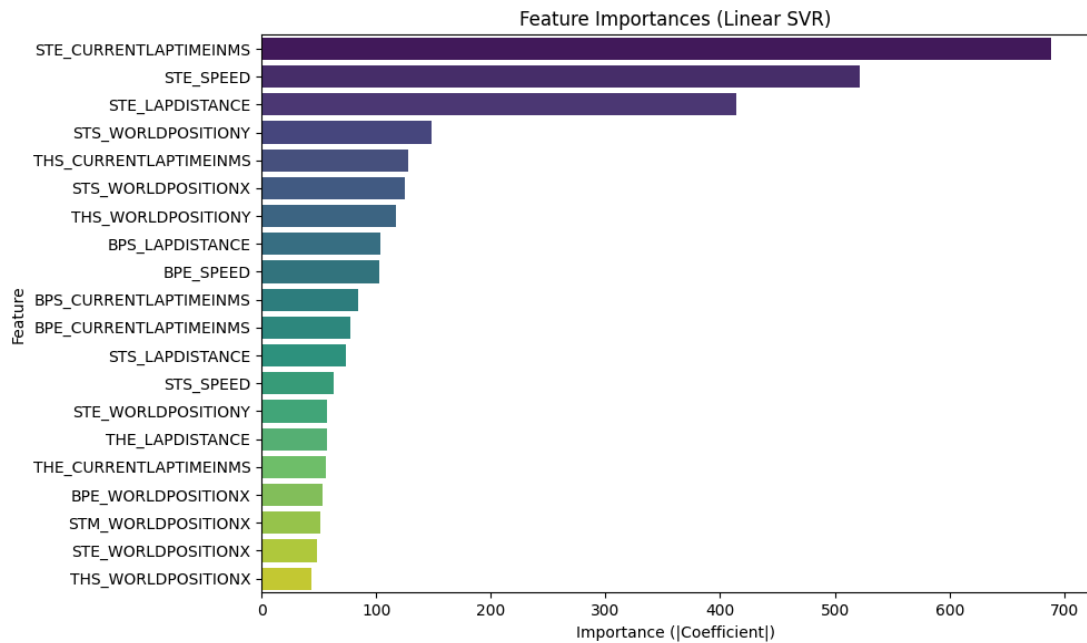


Figure 13: Top important features of Linear SVR

XGBoost Regression

XGBoost (eXtreme Gradient Boosting) is an ensemble learning algorithm that uses decision trees as its weak base learner, combining them together to form a strong learner through gradient boosting. This is when new trees are created based on the first tree's mistakes, using the residuals of the tree's predictions. XGBoost differs from regular gradient boosting because it offers built-in regularisation as part of the learning objective, correcting overfitting using both L1 (LASSO) Regularisation and L2 (Ridge) Regularisation. It is also able to handle missing values, and due to its cache-aware prefetching algorithm, it is able to reduce the runtime on large datasets.

XGBoost was seen in 'AI-enabled prediction of sim racing performance using telemetry data' by Fazilat Hojaji, Adam J. Toth, John M. Joyce, Mark J. Campbell (2024), where it outperformed other classifiers in the same paper. These classifiers were trained for performance prediction, and therefore assist in optimisation. It is important to note that the goal in the paper was to predict the difference in slow and fast laps, and the behaviours within those laps that caused it to be that speed. Our goal is a little different - ensuring that we have a predictor that can identify ideal behaviours to eventually obtain a fast lap through optimal navigation of the Melbourne Albert Park chicane.

For this model, it was found that removing the upper IQR 'CURRENTLAPTIMEINMS' of BPS, BPE, STE, and THS improved the MSE the most, and as a result, improved all other metrics compared to the original dataset. After that, hyperparameter tuning was performed, and the best parameters were: {'subsample': 0.9, 'n_estimators': 300, 'max_depth': 6, 'learning_rate': 0.12, 'colsample_bytree': 0.8}. It is also important to note that even though XGBoost has the ability to run feature selection, all of the features that are in the model need to be optimized - it is not ideal to choose the best features, as these will all need to be considered later.

Hyperparameter Tuning?	MSE	RMSE	MAE	Adjusted R ²
Before	37816.44	194.46	129.64	0.8490
After	35975.91	189.67	127.31	0.8562

Table 2: Results for XGBoost Regression before and after hyperparameter testing

LightGBM

LightGBM is a tree-based gradient boosting model designed to optimize speed, scalability, and performance. Unlike other boosting models, LightGBM grows trees leaf-wise, expanding the leaf with the largest error at each step. This focuses model capacity on the areas with the biggest mistakes and helps the algorithm handle large datasets efficiently. LightGBM also includes several built-in regularisation mechanisms, like 'min_data_in_leaf', 'num_leaves', and 'max_depth', which can help reduce overfitting. However, the model can still overfit on small datasets, so it performs best on clean, well-prepared data.

We selected LightGBM as one of our models because the article 'Predicting F1 Lap Times: A Comparison of ML Models' (Babbar, 2024) showed that tree-based methods outperformed traditional models in both accuracy and computational efficiency. Its leaf-wise tree-growing strategy is especially effective at capturing localised, nonlinear patterns in telemetry data, which is crucial in motorsport applications where small behaviour changes can significantly affect lap time. LightGBM also scales well to large datasets and handles complex feature interactions efficiently, making it a strong and practical choice for our modelling pipeline.

We ran the model on the cleaned dataset and achieved an RMSE of about 0.266 seconds, which is a strong result. But because small time differences have a huge impact in R1, we carried out hyperparameter tuning to search for the optimal parameters. The best parameters found by 'GridSearchCV' were: {'colsample_bytree': 0.8, 'learning_rate': 0.01, 'max_depth': 8, 'min_child_samples': 20, 'n_estimators': 1000, 'num_leaves': 31, 'reg_alpha': 0.1, 'reg_lambda': 0.1, 'subsample': 0.8}. As shown by the final performance metrics below (Figure 14), the tuned model was slightly more stable but its performance was limited by the potential noise in the data. Overall, the LightGBM model delivers a strong predictive performance for lap time using driver-controlled features. Further gains are likely to come from additional filtering of noise as opposed to further hyperparameter tuning.

```
MSE : 70,698.12
RMSE: 265.89
R2 : 0.8236
Adj R2: 0.6794
MAE : 166.61
```

Figure 14: Performance Metrics of the final LightGBM model

Model Results

The table below summarises the performance of each model used in our project.

	MSE	RMSE	Adjusted R^2	MAE
Linear Regression	2915701.85	1707.54	0.3247	982.52
Elastic Net Regression	1987021.49	1409.62	0.5398	879.41
KNNR	219623.44	468.64	0.0372	312.08
Random Forest	116485.87	341.30	0.8178	192.58
SVR	9718.65	98.58	0.9239	75.68
XGBoost	35975.91	189.67	0.8564	127.31
LightGBM	70698.12	265.89	0.6794	166.61

Table 3: Comparison of model Performance across Various Metrics

Our baseline Linear Regression model performed poorly, with a high MSE (2915701.85) and a low adjusted R^2 (0.5983). Elastic Net Regression improved on this slightly, with a lower MSE (1987021.49) and a higher adjusted R^2 (0.6649). However, compared to the other models, these regression models both performed poorly. KNN demonstrated substantial improvement over linear regression in terms of MSE, RMSE and MAE. However, its adjusted R^2 was considerably lower, explaining only 3% of the variance in lap time. Tree-based models, including Random Forest, XGBoost and LightGBM, outperformed the aforementioned models across all metrics, achieving notably lower RMSE values and substantially higher adjusted R^2 scores. Across these tree-based models, XGBoost performed the best. The Support Vector Regression (SVR) model displays the strongest overall performance, with substantially lower error metrics and a clear margin of accuracy compared to any other model. Consequently, SVR was selected as our final model.

Feature Optimization

With our final model selected, we focused on the main objective of our project: identifying the optimal driver trigger points and input values for navigating the chicane to minimise time and maximise exit speed. To achieve this, we used the 'differential_evolution' function from the SciPy package. This algorithm searches for the global minimum of a multivariate function. In our case, the function leverages insights from SVR - data on driver input and lap time - to identify the conditions that produce the quickest exit from the chicane.

To use this function, we had to set the bounds for each variable, to ensure the function only explored possible ranges. Whilst performing this investigation, it was important to ensure that:

1. The model gave a possible combination of key trigger points and their characteristics
2. The model gave a realistic output time

For our first attempt, we set each feature's bounds to its minimum and maximum values, which produced a predicted lap time of 200ms, which is clearly impossible. We then set the bounds to lie between the track's left and right boundaries but this approach still produced an unrealistic lap time. For our third attempt, we applied a specific order to the key trigger points to ensure the model preserved their order (as shown below). However this attempt also produced an unreasonably low lap time.

*Braking Start (BPS) < Throttle End (THE) < Steering Start (STS) < Braking End (BPE) < Throttle Start (THS)
< Steer Midpoint (STM) < Steer End (STE)*

The core reason these attempts were unsuccessful is that 'differential_evolution' does not distinguish between realistic and unrealistic outputs - its sole goal is to find the global minimum. In doing so, the function tests the absolute limits of the model without regard for practicality. Although constraints were applied to the model, they were insufficient in preventing the algorithm from producing unrealistic lap times.

Through trial and error, we determined that setting the bounds to the 25th and 75th percentile range produced a target lap time that was both realistic and competitive. To ensure the generated lap was reasonable, we used K-NN to find the closest actual lap from the dataset and compared their feature values. As shown in Appendix Figure 5, the differences between the generated lap and the actual lap were minor, confirming that the algorithm generated lap was realistic.

Recommended Lap

Our suggested optimal lap navigation would result in a time of 11266.63ms, or around 11.27 seconds from the beginning of the race to the target distance of 900m into the lap - past the end of the chicane. The key findings can be found in Table 4, whilst a complete table of all input values from the model can be found in Appendix Figure 6. In Figure 15, a plot can be found containing the key trigger points for both the optimal lap and the closest existing lap for comparison.

The optimal first braking point (BPS), or throttle off point (THE) should lie between coordinates (304.40, 287.14) to (306.62, 291.51), where the speed before this point should be around 319km/h, at a current lap time of 2897-3118ms.

The steer starting point (STS), where the car begins to steer for navigating Turn 1, the model suggests that it should occur at (345.96, 258.60), at a speed of 274km/h, at around 3760ms.

The foot should be lifted off the brake pad (BPE) at (337.63, 217.12), where the speed gets to around 172km/h, with a current lap time of 3714ms. Promptly after, the throttle should be reapplied (THS) at (364.24, 198.82), when the speed drops to 152km/h, at a lap time of 4429ms.

The midpoint of steering (STM) should then occur at (373.76, 155.46), at a speed of 175km/h, at a time of 5616ms. Finally, the steering wheel should then straighten up (STE) and prepare to travel down the straight between Turn 2 and Turn 3 at (398.69, 10.63), with speed 241km/h at a time of 8284ms.

The steer starting point (STS) and brake end point (BPE) are located at the same section of the track, but on opposite sides. This is not realistic, and highlights there are some underlying issues with the values that should be worked around in the future. In comparison to the closest lap features, STS should occur before BPE. It is also important to note that some of the key trigger points also lay outside of the track bounds, but with Formula 1 standards, as long as one wheel lies on the track at all times, these points may be allowed for navigation of the chicane, and therefore maximal exit speed of the chicane.

	BPS	THE	STS	BPE	THS	STM	STE
Speed	319 km/h	319 km/h	274 km/h	172 km/h	152 km/h	175 km/h	241 km/h
Time	3118 ms	2897 ms	3760 ms	3714 ms	4429 ms	5616 ms	8284 ms
Position X	306.62	304.40	345.96	337.63	364.24	373.76	398.69
Position Y	291.51	287.14	258.60	217.12	198.82	155.46	10.63

Table 4. Summary of the key findings from differential evolution

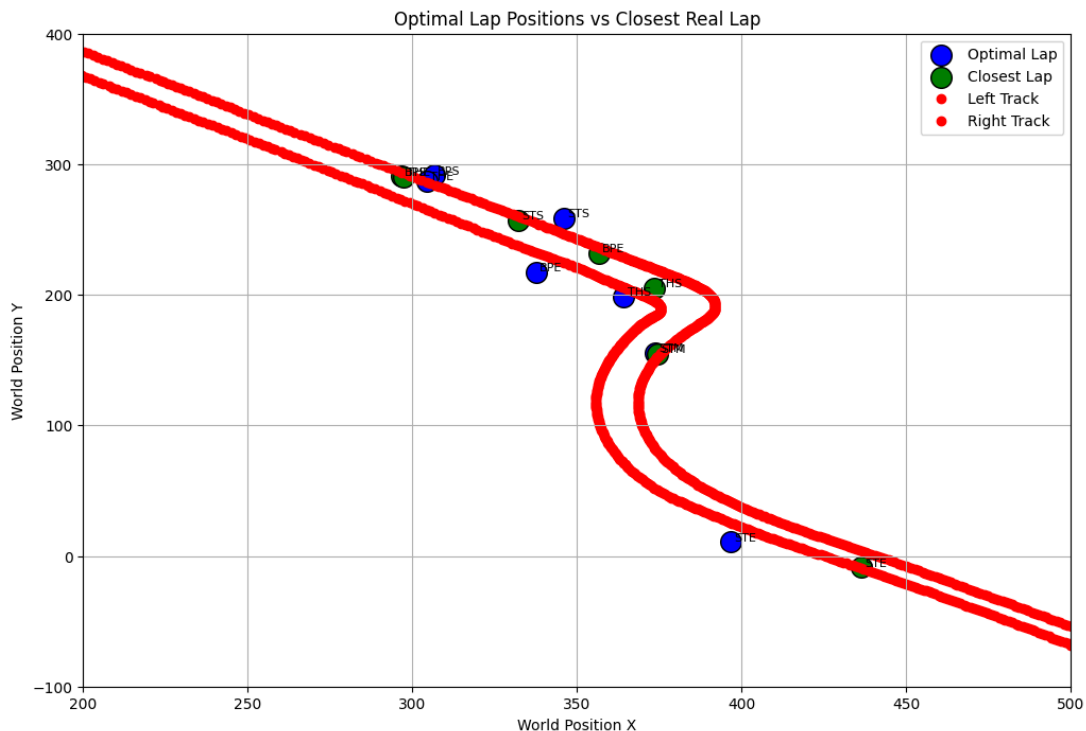


Figure 15. Plot of the Optimal Lap features (Blue) and Closest Lap features (Green)

Limitations

Despite the project's success in identifying an optimal path for drivers to minimize their time and maximise their speed when exiting the chicane, there were several limitations that restricted the depth and generalisation of our findings.

Feature engineered dataset

All models were trained on a dataset that had undergone a substantial amount of feature engineering, and so, all of our insights depend on how well the engineered features capture real driving behaviour. As a result, the interpretability and reliability of our models are constrained by the accuracy of the dataset provided by the other team.

Aggregated laps

The data was aggregated across multiple segments and laps, meaning the tested models did not have access to continuous lap information. As a result, the models only captured driver behaviour at key reference points rather than across the lap. So, a driver may reach key points with optimal telemetry values but arrive there suboptimally due to earlier inefficiencies in braking, entry or track positioning. Therefore, while our model predicts good performance based on key point features, the overall lap time may not show the expected improvement.

Optimization Limitations

Due to the differential algorithm's inability to distinguish between realistic and unrealistic outputs, during optimization, the algorithm's bounds were determined using trial and error. Therefore, it is possible that alternative bounds could produce a reasonable set of driver inputs capable of achieving a faster chicane time.

Furthermore, as noted in the recommendations, there were issues with our algorithm generated lap. Notably, the steering start point and braking end point were located on opposite sides of the same section, which is technically not possible. Therefore, without adjustment, our recommended lap may contain inefficiencies and be infeasible in practice.

Real-world Applications

The dataset does not capture contextual information such as tyre wear, track temperature or weather which are important in real-world racing. Therefore, the model's definition of "optimal" performance is limited to driver behaviour, and its predictions may not be applicable outside of the simulation.

Conclusion

SVR was identified as the strongest model for capturing the relationship between driver inputs and performance through the chicane between Turns 1 and 2 of the Albert Park Circuit. Using insights from the model combined with differential evolution, we generated a realistic lap that closely mirrors driver movements in the dataset, thus providing a potential path to effectively navigate the chicane. While our findings are limited by the engineered, segment-based dataset, our approach offers a robust behavioural framework for identifying optimal ranges of driver inputs that will improve performance.

Authors: Prasamsa Bhusal (z5477303), Jacinta Peng (z5482587), Charlotte Fang Hendro (z5363431), Tay Hui Qi (z5473860), Hui Jun Ng (z5480075)

Github Link: <https://github.com/saaamsaaa/F1-Modelling-Group>

The Github above contains all of the code that was used throughout our project.

References

- Abulkhair, A 2021, *Best n _estimators for RandomForest*, Kaggle, accessed 8th November 2025, <<https://www.kaggle.com/code/ahmedabdulhamid/best-n-estimators-for-randomforest>>.
- Adam J. Toth, John M. Joyce, Mark J. Campbell 2024, 'AI-enabled prediction of sim racing performance using telemetry data', *Computers in Human Behavior Reports*, Volume 14. <https://www.sciencedirect.com/science/article/pii/S2451958824000472>
- Babbar, T 2024, *Predicting F1 Lap Times: A Comparison of ML Models*, Medium, accessed 14th November 2025, <<https://medium.com/@tapanbabbar/predicting-f1-lap-times-a-comparison-of-ml-models-275ac8a06e19>>.
- Carter, D 2025, *FP2 Australia telemetry analysis: key details of thrilling Ferrari-McLaren battle*, Scuderia Fans, accessed 10th November 2025, <<https://scuderiafans.com/fp2-australia-telemetry-analysis-key-details-of-thrilling-ferrari-mclaren-battle/>>.
- El Haber, Elias, et al., 2025, *Formula 1 Race Winner Prediction Using Random Forest and SHAP Analysis.*, Accessed 15 11 2025, <<https://hal.science/hal-05148972/document>>
- Esri n.d., *How LightGBM algorithm works*, Esri, accessed 10th November 2025, <<https://pro.arcgis.com/en/pro-app/latest/tool-reference/geoai/how-lightgbm-works.htm>>.
- Frost, J 2025, *How to Interpret Adjusted R-Squared and Predicted R-Squared in Regression Analysis*, Statistics By Jim, accessed 31st October 2025, <<https://statisticsbyjim.com/regression/interpret-adjusted-r-squared-predicted-r-squared-regression/>>
- Frost, J 2025, *Mean Absolute Error (MAE)*, Statistics By Jim, accessed 31st October 2025, <<https://statisticsbyjim.com/glossary/mean-absolute-error/>>.
- Frost, J 2025, *Mean Squared Error (MSE)*, Statistics By Jim, accessed 31st October 2025, <<https://statisticsbyjim.com/regression/mean-squared-error-mse/>>.
- Frost, J 2025, *Root Mean Squared Error (RMSE)*, Statistics By Jim, accessed 31st October 2025, <<https://statisticsbyjim.com/regression/root-mean-square-error-rmse/>>.
- Geeks For Geeks 2025, *K-Nearest Neighbor(KNN) Algorithm*, Geeks For Geeks, accessed 6th November 2025, <<https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>>
- Geeks For Geeks 2025, *Support Vector Regression (SVR) using Linear and Non-Linear Kernels in Scikit Learn*, Geeks For Geeks, accessed 8th November 2025, <<https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>>.
- Kavlakoglu, E & Russi, E n.d., *What is XGBoost*, IBM, accessed 7th November 2025, <<https://www.ibm.com/think/topics/xgboost>>.
- Kavlakoglu, E n.d., *What are support vector machines (SVMs)?*, IBM, accessed 6th November 2025, <<https://www.ibm.com/think/topics/knn>>.
- Kavlakoglu, E n.d., *What is the k-nearest neighbors (KNN) algorithm?*, IBM, accessed 6th November 2025, <<https://www.ibm.com/think/topics/knn>>.
- Nvidia 2025, *XGBoost*, Nvidia, accessed 7th November 2025, <<https://www.nvidia.com/en-au/glossary/xgboost/>>.

Remonda A, Veas E, Luzhnica G 2021, 'Comparing driving behavior of humans and autonomous driving in a professional racing simulator', PLoS ONE, Volume 16(2), accessed 28th October 2025, <<https://doi.org/10.1371/journal.pone.0245320>>.

Shafi, A 2025, *Random Forest Classification in Python With Scikit-Learn: Step-by-Step Guide (with Code Examples)*, DataCamp, accessed 10th November 2025, <<https://www.datacamp.com/tutorial/random-forests-classifier-python>>.

The Click Reader 2021, *Random Forest Regression Explained with Implementation in Python*, Medium, accessed 10th November 2025, <<https://medium.com/@theclickreader/random-forest-regression-explained-with-implementation-in-python-3dad88caf165>>.

Tonoli, A 2024, 'Data-Driven Vehicle Performance Optimization for Formula Student Racing' Master's Thesis, Politecnico di Torin.

Tyagi, A 2025, *What is XGBoost Algorithm?*, Analytics Vidhya, accessed 10th November 2025, <<https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/>>.

Appendix

Figure 1. A comparative boxplot of throttle values between throttle start (THS) and throttle end (THE).

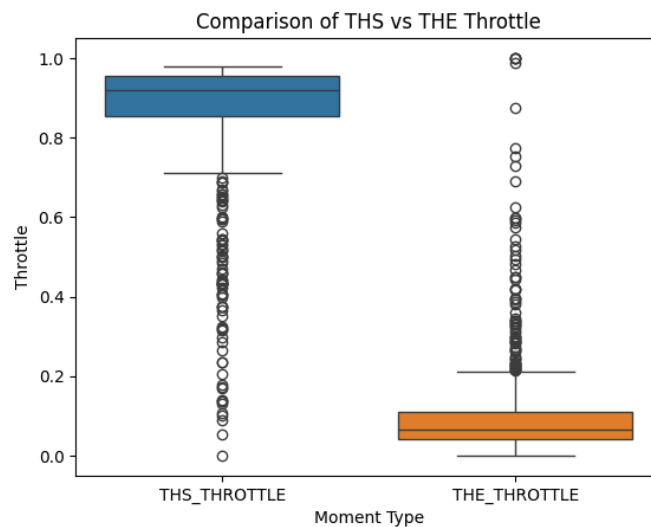


Figure 2. A comparative boxplot of brake values between throttle start (THS) and throttle end (THE).

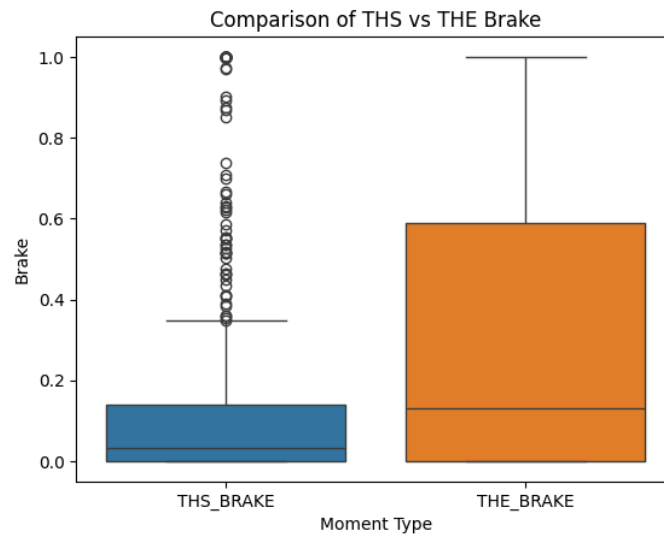


Figure 3. A comparative boxplot of speed values between throttle start (THS) and throttle end (THE).

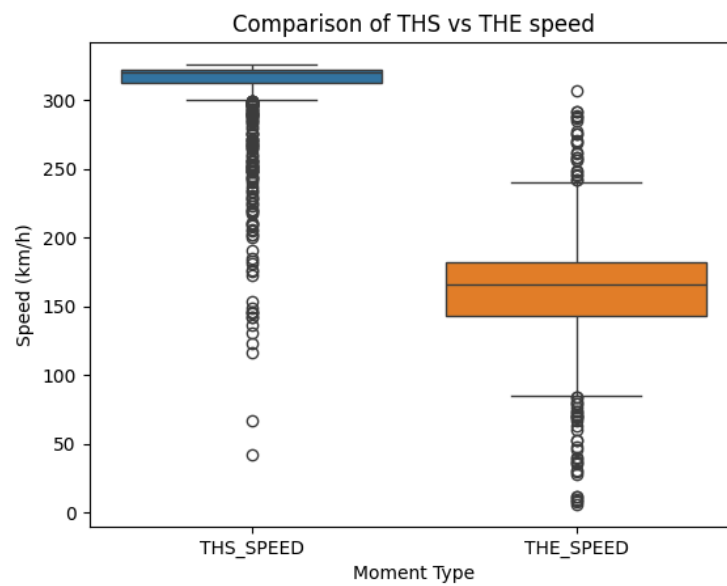


Figure 4. A comparative boxplot of steering angle values between throttle start (THS) and throttle end (THE).

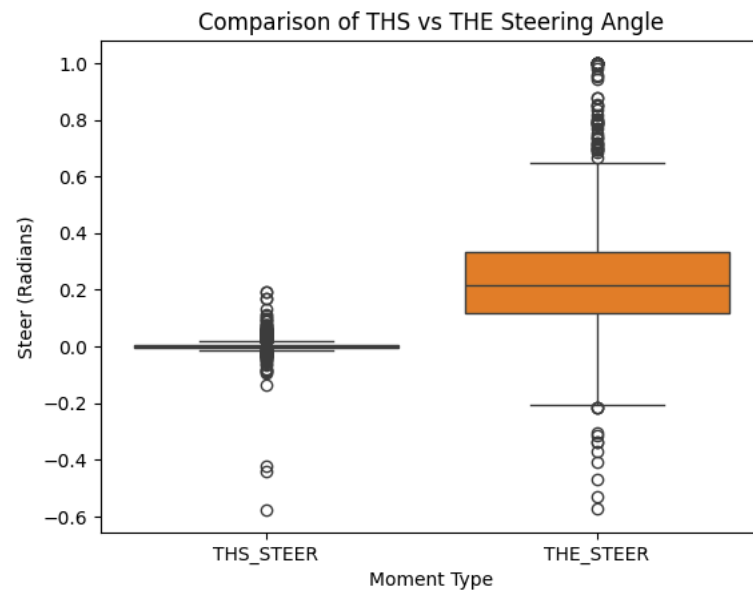


Figure 5. Difference in driver inputs between the algorithm generated lap and actual lap from the dataset.

Note that the lap time is in milliseconds, so although the large number, the difference between the two laps is very minor across all features.

```

BPS_SPEED: -2.908701547667249
BPS_THROTTLE: 0.05131285210335923
BPS_STEER: 0.002506903317265175
BPS_BRAKE: 0.0
BPS_CURRENTLAPTIMEINMS: 202.10708655949247
BPS_LAPDISTANCE: -6.0
BPS_WORLDPOSITIONX: 9.936738314636614
BPS_WORLDPOSITIONY: 0.6377671514462122
BPE_SPEED: -36.0
BPE_THROTTLE: 0.0
BPE_STEER: 0.14343954438489348
BPE_BRAKE: 0.021199854666713036
BPE_CURRENTLAPTIMEINMS: -314.27946414848293
BPE_LAPDISTANCE: -11.184332953028786
BPE_WORLDPOSITIONX: -18.99618769407857
BPE_WORLDPOSITIONY: -14.843050377524008
THS_SPEED: -13.678508866254049
THS_THROTTLE: -0.014824817800162954
THS_STEER: 0.04997593439966869
THS_BRAKE: 0.5596521074072167
THS_CURRENTLAPTIMEINMS: -221.82174386320276
THS_LAPDISTANCE: 5.777834642557025
THS_WORLDPOSITIONX: -9.350148742467297
THS_WORLDPOSITIONY: -6.132358377357917
THE_SPEED: -3.0
THE_THROTTLE: -0.006129669143076111
THE_STEER: -0.0039966352501876745
THE_BRAKE: -0.007420179579121724
THE_CURRENTLAPTIMEINMS: -29.27523314351083
THE_LAPDISTANCE: 19.0
THE_WORLDPOSITIONX: 6.991406457048811
THE_WORLDPOSITIONY: -3.041396501677525
STS_SPEED: 4.0
STS_THROTTLE: 0.0
STS_STEER: -0.00023130205610647457
STS_BRAKE: -0.04603799065595893
STS_CURRENTLAPTIMEINMS: 260.7763126721852
STS_LAPDISTANCE: 20.0
STS_WORLDPOSITIONX: 13.79180334469686
STS_WORLDPOSITIONY: 1.5447260100306721
STM_SPEED: -8.470658102278861
STM_THROTTLE: 0.0
STM_STEER: -1.1635657715114434e-15
STM_BRAKE: 0.0
STM_CURRENTLAPTIMEINMS: -104.68045180036825
STM_LAPDISTANCE: 1.4126046818415716
STM_WORLDPOSITIONX: -0.6750340037296496
STM_WORLDPOSITIONY: 0.5182763079628216
STE_SPEED: -8.0
STE_THROTTLE: 0.0
STE_STEER: -2.8015784137025435e-16
STE_BRAKE: 0.0
STE_CURRENTLAPTIMEINMS: -516.9718466795603
STE_LAPDISTANCE: -7.522510507652896
STE_WORLDPOSITIONX: -39.58817952302951
STE_WORLDPOSITIONY: 19.345268366677224

```

Figure 6. The full set of inputs suggested by the algorithm generated lap

	BPS	THE	STS	BPE	THS	STM	STE
Speed	319 km/h	319 km/h	274 km/h	172 km/h	152 km/h	175 km/h	241 km/h
Throttle	1.00	0.89	0.00	0.00	0.11	1.00	1.00
Brake	0.00	0.004	0.76	0.90	0.59	0.00	0.00
Steer	0.00	0.00	0.00	0.23	0.35	0.00	0.00
Time	3118 ms	2897 ms	3760 ms	3714 ms	4429 ms	5616 ms	8284 ms
Lap Distance	249m	275m	324m	327m	375m	428m	610m
Position X	306.62	304.40	345.96	337.63	364.24	373.76	398.69
Position Y	291.51	287.14	258.60	217.12	198.82	155.46	10.63